Diffusion Models

1.1 Score Matching

Score matching objective

Song and Ermon introduce score matching with annealed Langevin dynamics [20]. Let $U_{\text{data}}(x) = \nabla_x \ln p_{\text{data}}(x)$ denote the (unknown) score of the data and let $s_\theta(x)$ denote a parametric *score network*, trained to approximate $U_{\text{data}}(x)$. The parameters θ are estimated by minimizing:

$$\begin{aligned} L(\theta) &= \frac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}}\left\{||s_\theta(x) - U_{\text{data}}(x)||^2\right\} \\ &= \mathbb{E}_{x\sim p_{\text{data}}}\left[\text{tr}\{\nabla_x s_\theta(x)\} + \frac{1}{2}||s_\theta(x)||_2^2\right] + C \end{aligned}$$

Langevin dynamics

Langevin dynamics is a method of producing samples from $p(x)$ when the score function $U(x) = \nabla_x \ln p(x)$ is known. Let $x_0 \sim \pi(x)$ denote a sample from a prior distribution. Then for fixed $\epsilon > 0$ and $t \in \{1, \ldots, T\}$, recursively calculate:

$$x_t = x_{t-1} + \frac{\epsilon}{2}U(x_{t-1}) + \sqrt{\epsilon}z_t,$$

where $z_t \sim \mathcal{N}(0, I)$. Under regularity conditions, for $\epsilon \to 0$ and $T \to \infty$, the distribution of x_T converges in law to $p(x)$. The continuous time stochastic differential equation (SDE) underlying Langevin dynamics is:

$$dX_t = \underbrace{\frac{1}{2}\nabla_x \ln p(X_t)dt}_{\text{drift}} + dW_t,$$

where X_t is the random state at time t and W_t is standard Brownian motion. The drift term pushes the particle toward regions of higher probability density, while the Brownian increments prevent the particle from collapsing to a local mode.

Noise conditional score networks

Let $(\sigma_l)_{l=1}^L$ denote a decreasing geometric sequence of noise levels, with $\sigma_l = \sigma_1 r^{l-1}$ for some $r \in (0, 1)$. Let $q_\sigma(\tilde{x}) = \int q_\sigma(\tilde{x}|x)p_{\text{data}}(x)dx$ denote the perturbed data distribution, where $q_\sigma(\tilde{x}|x) = \mathcal{N}(\tilde{x} \mid x, \sigma^2 I)$. The noise conditional score network (NCSN) $s_\theta(\tilde{x}, \sigma)$ aims to approximate $\nabla_{\tilde{x}} \ln q_\sigma(\tilde{x})$ for corrupted input $\tilde{x}$ and noise-level σ. For a given noise level, the denoising objective is:

$$\ell(\theta; \sigma) = \frac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}, \tilde{x}\sim q_\sigma(\tilde{x}|x)}\left|\left|s_\theta(\tilde{x}, \sigma) - \nabla_{\tilde{x}} \ln q_\sigma(\tilde{x}|x)\right|\right|^2$$

Notice that the NCSN targets the marginal score $\nabla_{\tilde{x}} \ln q_\sigma(\tilde{x})$ but that the objective matches $s_\theta(\tilde{x}, \sigma)$ against the conditional score $\nabla_{\tilde{x}} \ln q_\sigma(\tilde{x}|x)$. Use of the conditional score in the objective is a computational expedient. The optimized NCSN nevertheless satisfies:

$$s_{\theta^*}(\tilde{x}, \sigma) = \nabla_{\tilde{x}} \ln q_\sigma(\tilde{x}) = \mathbb{E}\left\{\nabla_{\tilde{x}} \ln q_\sigma(\tilde{x}|x) \mid \tilde{x}\right\}$$

The final objective is a weighted sum across noise levels:

$$\mathcal{L}(\theta; \sigma_1, \ldots, \sigma_L) = \frac{1}{L} \sum_{l=1}^{L} \lambda(\sigma_l) \ell(\theta; \sigma_l),$$

where $\lambda(\sigma_l)$ is a weight function chosen empirically to be $\lambda(\sigma_l) = \sigma_l^2$.

Training

Algorithm 1 Training loop for NCSN

Require: Noise levels $(\sigma_l)_{l=1}^L$, weights $\lambda(\sigma_l)$, score network s_θ, batch size B

for $k = 1, \ldots, K$ **do**
  Sample a batch from p_{data}, initialize L.
  for $l = 1, \ldots, L$ **do**
    Sample noise from $\mathcal{N}(0, I)$.
    Create corrupted observations $\tilde{x}^{(i)} \leftarrow x^{(i)} + \sigma_l z^{(i)}$.
    Score-matching loss $L \leftarrow L + \frac{\lambda(\sigma_l)}{2B} \sum_{i=1}^{B} \left\| s_\theta(\tilde{x}^{(i)}, \sigma_l) + \frac{\tilde{x}^{(i)} - x^{(i)}}{\sigma_l^2} \right\|^2$.
  end for
  Update θ using ∂L.
end for

Annealed Langevin dynamics

Fix a base step size $\epsilon > 0$, a sequence of noise levels $\sigma_1 > \cdots > \sigma_L$, and the number of steps per noise level T. Then, for $l \in \{1, \ldots, L\}$, set $\alpha_l \leftarrow \epsilon \sigma_l^2 / \sigma_L^2$, and make T updates:

$$\tilde{x}_t \leftarrow \tilde{x}_{t-1} + \frac{\alpha_l}{2} s_\theta(\tilde{x}_{t-1}, \sigma_l) + \sqrt{\alpha_l} z_t,$$

where $z_t \sim \mathcal{N}(0, I)$. The final $\tilde{x}_T$ at level l serves as the sample from q_{σ_l} and as the initializer for sampling $q_{\sigma_{l+1}}$. This procedure is used only during inference.

1.2 Denoising Diffusion Probabilistic Models

Ho *et al.* introduce DDPM [8]. Let x_0 denote the observed data, and let $x_1, \ldots, x_T$ denote a sequence of latents generated from x_0 by a known forward Markov noising process:

$$q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1}).$$

At each step, noise is added to x_{t-1} to generate x_t:

$$x_t = \sqrt{1 - \beta_t} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \epsilon_t,$$

where $\epsilon_t \sim \mathcal{N}(0, I)$ and the β_t define the *noise schedule*. After many steps, the distribution of x_T is approximately $\mathcal{N}(0, I)$. The reverse process is factorized as:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t),$$

where $p(x_T) = \mathcal{N}(0, I)$ and $p_\theta(x_{t-1}|x_t) = \mathcal{N}\{\mu_\theta(t, x_t), \Sigma_\theta(t, x_t)\}$. The parameters θ of the learned generative reverse process are estimated via variational inference. The negative evidence lower bound, used as a loss, is expressed in terms of KL divergences:

$$\begin{aligned} L_{\text{VLB}} &= L_0 + L_1 + \cdots + L_{T-1} + L_T, \\ L_0 &= -\ln p_\theta(x_0|x_1), \\ L_{t-1} &= \text{KL}\{q(x_{t-1}|x_t, x_0) \,\|\, p_\theta(x_{t-1}|x_t)\}, \\ L_T &= \text{KL}\{q(x_T|x_0) \,\|\, p(x_T)\}. \end{aligned}$$

With fixed reverse variances $\Sigma_\theta(t, x_t) = \sigma_t^2 I$, the KL terms from the ELBO simplify to weighted squared error objectives. Specifically, θ can be estimated by minimizing terms of the form:

$$L_{t-1}(\theta) = \frac{1}{2\sigma_t^2}\mathbb{E}_{(x_0,x_t)\sim q}\left\{||\mu_{t-1}(x_t, x_0) - \mu_\theta(t, x_t)||^2\right\} + C,$$

where $\mu_{t-1}(x_t, x_0) = \mathbb{E}_{x_{t-1}\sim q}(x_{t-1}|x_t, x_0)$. Through a reparameterization of $\mu_\theta(t, x_t)$ in terms of $\epsilon_\theta(t, x_t)$, minimizing this objective is shown to be equivalent to predicting ϵ_t in the marginal forward process:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Specifically, θ is estimated by minimizing:

$$L_{t-1}(\theta) = w_t \cdot \mathbb{E}_{x_0,\epsilon_t}\left\{||\epsilon_\theta(t, x_t) - \epsilon_t||^2\right\} + C,$$

where w_t is a time step-dependent weight.

1.3 Denoising Diffusion Implicit Models

Song *et al.* introduce DDIM [19]. Given data from a distribution $q(x_0)$, the goal is to learn an approximation $p_\theta(x_0)$ that is easy to sample from. DDPM is ultimately trained on the objective of predicting ϵ_t in:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t.$$

This task implies a specific form for the marginal distribution $q(x_t|x_0)$, but not for the full joint distribution $q(x_{1:T}|x_0)$. DDIM introduces a new family of joint distributions $q_\sigma(x_{1:T}|x_0)$ that have the same marginals but a different dependence structure across time. Specifically, the joint distribution is factorized as:

$$q_\sigma(x_{1:T}|x_0) = q_\sigma(x_T|x_0) \prod_{t=2}^{T} q_\sigma(x_{t-1}|x_t, x_0).$$

The forms of $q_\sigma(x_T|x_0)$ and $q_\sigma(x_{t-1}|x_t, x_0)$ are chosen to preserve $q(x_t|x_0)$. A new variational objective $J(\epsilon_\theta)$ is proposed and shown to be equivalent to:

$$J(\epsilon_\theta) = \sum_{t=1}^{T} \gamma_t(\sigma)\mathbb{E}_{x_0,\epsilon_t}\left\{||\epsilon_\theta(t, x_t) - \epsilon_t||^2\right\} + C,$$

where $\gamma_t(\sigma)$ is a set of weights. Using x_t, the uncorrupted sample can be predicted as:

$$\hat{x}_0^{(t)} = f_\theta^{(t)}(x_t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta^{(t)}(x_t)}{\sqrt{\bar{\alpha}_t}}.$$

Next, the reverse transition process can be sampled via:

$$p_\theta^{(t)}(x_{t-1}|x_t) = q_\sigma(x_{t-1}|x_t, \hat{x}_0^{(t)}),$$

for $t > 1$. When $\sigma_t = 0$, the reverse process is deterministic given x_T. This regime is called *implicit* because $p_\theta(x_0)$ is sampled by pushing $x_T \sim \mathcal{N}(0, I)$ through a deterministic map $x_0 = G_\theta(x_T)$ rather than by explicitly sampling from a closed-form density.

Algorithm 2 DDIM sampling on a subsequence

Require: Subsequence $\tau_1 < \cdots < \tau_K$, noise schedule $(\bar{\alpha}_t)$, model ϵ_θ, transition scales $\sigma_{t\to s}$

1: Sample $x_{\tau_K} \sim \mathcal{N}(0, I)$
2: **for** $k = K, \ldots, 2$ **do**
3: Set $t \leftarrow \tau_k$ and $s \leftarrow \tau_{k-1}$
4: $\hat{\epsilon}_t \leftarrow \epsilon_\theta(x_t, t)$
5: $\hat{x}_0^{(t)} \leftarrow \dfrac{x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}_t}{\sqrt{\bar{\alpha}_t}}$
6: Sample $z \sim \mathcal{N}(0, I)$
7: $x_s \leftarrow \sqrt{\bar{\alpha}_s}\hat{x}_0^{(t)} + \sqrt{1-\bar{\alpha}_s - \sigma_{t\to s}^2}\,\hat{\epsilon}_t + \sigma_{t\to s} z$
8: **end for**
9: Return $\hat{x}_0^{(1)} \leftarrow \dfrac{x_1 - \sqrt{1-\bar{\alpha}_1}\epsilon_\theta(x_1, 1)}{\sqrt{\bar{\alpha}_1}}$

Connection to differential equations

When $\sigma_t = 0$, the DDIM update can be written as:

$$\frac{x_s}{\sqrt{\bar{\alpha}_s}} = \frac{x_t}{\sqrt{\bar{\alpha}_t}} + \left(\sqrt{\frac{1-\bar{\alpha}_s}{\bar{\alpha}_s}} - \sqrt{\frac{1-\bar{\alpha}_t}{\bar{\alpha}_t}}\right)\epsilon_\theta^{(t)}(x_t).$$

Set $s = t - \Delta t$ and make the change of variables:

$$\bar{x}(t) = \frac{x(t)}{\sqrt{\bar{\alpha}_t}}, \qquad \sigma(t) = \sqrt{\frac{1-\bar{\alpha}(t)}{\bar{\alpha}(t)}}.$$

then the update becomes:

$$\bar{x}(t-\Delta t) - \bar{x}(t) = \{\sigma(t-\Delta t) - \sigma(t)\}\epsilon_\theta^{(t)}\left(\frac{\bar{x}(t)}{\sqrt{1+\sigma^2(t)}}\right).$$

Finally, dividing both sides by $-\Delta t$ and taking the limit as $\Delta t \to 0$:

$$\frac{d\bar{x}(t)}{dt} = \frac{d\sigma(t)}{dt} \cdot \epsilon_\theta^{(t)}\left(\frac{\bar{x}(t)}{\sqrt{1+\sigma^2(t)}}\right).$$

The connection to differential equations allows the implicit sampling scheme to be viewed as applying Euler's method to solve a reverse time ODE that maps from noise at time $t = T$ to data at time $t = 0$.

1.4 Score-Based Generative Modeling

Song *et al.* [21] show that score matching with Langevin dynamics [20] and denoising diffusion probabilistic models [8] can be unified within a framework where data corruption is viewed as a continuous, forward-time stochastic differential equation (SDE) and sample generation as a corresponding reverse-time SDE.

DDPM is a score matching model

Starting from the simplified DDPM loss:

$$L_{\text{simple}} = \sum_{t=1}^{T} \mathbb{E}_{x_0 \sim p_{\text{data}},\, \epsilon \sim \mathcal{N}(0,I)} \left\| \epsilon - \epsilon_\theta(x_t, t) \right\|^2, \qquad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon.$$

Relabel $x \leftarrow x_0$, $\tilde{x} \leftarrow x_t$, and $\alpha_i \leftarrow \bar{\alpha}_t$. Then the perturbation kernel is:

$$p_{\alpha_i}(\tilde{x} \mid x) = \mathcal{N}\{\tilde{x}; \sqrt{\alpha_i}\, x, (1-\alpha_i) I\}.$$

The gradient w.r.t. the perturbed input is:

$$\nabla_{\tilde{x}} \log p_{\alpha_i}(\tilde{x} \mid x) = -\frac{\tilde{x} - \sqrt{\alpha_i} x}{1-\alpha_i} = -\frac{\epsilon}{\sqrt{1-\alpha_i}},$$

where $\epsilon \sim \mathcal{N}(0, I)$. Thus $\epsilon = -\sqrt{1-\alpha_i}\, \nabla_{\tilde{x}} \log p_{\alpha_i}(\tilde{x} \mid x)$. Now define:

$$s_\theta(\tilde{x}, i) \equiv -\frac{\epsilon_\theta(\tilde{x}, i)}{\sqrt{1-\alpha_i}},$$

such that $\epsilon_\theta(\tilde{x}, i) = -\sqrt{1-\alpha_i}\, s_\theta(\tilde{x}, i)$. Substituting into the DDPM loss yields:

$$\left\| \epsilon - \epsilon_\theta(\tilde{x}, i) \right\|^2 = (1-\alpha_i) \left\| s_\theta(\tilde{x}, i) - \nabla_{\tilde{x}} \log p_{\alpha_i}(\tilde{x} \mid x) \right\|^2.$$

Therefore:

$$L_{\text{simple}} = \sum_{i=1}^{N} (1-\alpha_i)\, \mathbb{E}_{x \sim p_{\text{data}},\, \tilde{x} \sim p_{\alpha_i}(\tilde{x}|x)} \left\| s_\theta(\tilde{x}, i) - \nabla_{\tilde{x}} \log p_{\alpha_i}(\tilde{x} \mid x) \right\|^2.$$

Score-based generative modeling with SDEs

The goal is to construct a diffusion process $\{X_t\}_{t=0}^{t=T}$ such that $X_0 \sim p_0$, the data distribution, and $X_T \sim p_T$, the prior distribution. The forward diffusion process can be modeled as the solution to an Ito SDE:

$$dX_t = f(X_t, t)dt + g(t)dW_t,$$

where $f(X_t, t)$ is a vectorial drift coefficient, $g(t)$ is a scalar diffusion coefficient, and W_t is Brownian motion. Let $p_t(x)$ denote the marginal density of X_t, and $p_{st}(x_t|x_s)$ a transition kernel from time s to time t. The reverse diffusion process satisfies:

$$dX_t = \{f(X_t, t) - g^2(t) \nabla_x \ln p_t(x)|_{x=X_t}\} dt + g(t) d\overline{W}_t,$$

where $\nabla_x \ln p_t(x)$ is the score of the marginal distribution for X_t, and $\overline{W}_t$ is centered, time-reversed Brownian motion. To approximate $\nabla_x \ln p_t(x)$, a time-dependent score model $s_\theta(x, t)$ is trained by minimizing:

$$\mathbb{E}_t \left[\lambda(t) \mathbb{E}_{X(0) \sim p_0} \mathbb{E}_{X(t) \sim p_{0t}(X(t)|X(0))} \left\{ \left\| s_\theta(X(t), t) - \nabla_{X(t)} \log p_{0t}(X(t) \mid X(0)) \right\|_2^2 \right\} \right]$$

with respect to θ. Here $\lambda(t)$ is a positive weight function, t is uniformly sampled over $[0, T]$, $X_0 \sim p_0(x)$, $X_t \sim p_{0t}(X_t|X_0)$.

Probability flow ODE

The *probability flow ODE* is a deterministic differential equation whose marginals match those of the corresponding SDE when the score is known exactly. After training a score model $s_\theta(x, t)$, samples can be generated by drawing $X_T \sim p_T$ from the prior, then deterministically integrating:

$$dX_t = \left\{ f(X_t, t) - \frac{1}{2} g^2(t) s_\theta(X_t, t) \right\} dt$$

backward in time. Conditional on X_T, the trajectory is deterministic; randomness enters only through the initial draw from the prior.

1.5 Improved DDPM

Reverse-process variance estimation

The original DDPM study [8] did not learn the reverse-process variance. Instead, the reverse transition was parameterized with fixed variance $\Sigma_\theta(x_t, t) = \sigma_t^2 I$, where σ_t^2 was regarded as a hyperparameter. Values of $\sigma_t^2 = \mathbb{V}(X_t | X_{t-1}) \equiv \beta_t I$, the variance of the forward process, and $\sigma_t^2 = \mathbb{V}(X_{t-1} | X_t, X_0) \equiv \tilde{\beta}_t I$ were explored and found to yield similar performance. Nichol and Dhariwal [17] observe that β_t and $\tilde{\beta}_t$ are almost equal except near $t = 0$, and that their similarity increases with the total number of diffusion steps T. They propose parameterizing the reverse-process variance as:

$$\operatorname{diag}\{\Sigma_\theta(x_t, t)\} = \exp\left\{ \operatorname{diag}(v) \ln \beta_t + (1 - v) \ln \tilde{\beta}_t \right\},$$

where v is a learned weight vector. Although the elements of v are not explicitly constrained to reside in the $[0, 1]$ interval, empirically v was found to act as an interpolant. The final *hybrid* objective takes the form:

$$L_{\text{Hybrid}} = L_{\text{Simple}} + \lambda \cdot L_{\text{VLB}}$$

where L_{Simple} is the simplified MSE objective proposed by Ho *et al.* [8], which does not depend on $\operatorname{diag}\{\Sigma_\theta(x_t, t)\}$, and L_{VLB} is the variational lower bound objective, which does. A stop-gradient is applied to the mean/noise-prediction output in L_{VLB}, such that L_{Simple} supervises $\mu_\theta(x_t, t)$ while L_{VLB} supervises $\Sigma_\theta(x_t, t)$.

Cosine noise schedule

The linear schedule for β_t adopted in DDPM causes the signal coefficient $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ appearing in the forward sampling expression:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t$$

to decay super-linearly towards zero, which in turn causes the forward-process to approach noise too quickly. Nichol and Dhariwal [17] propose a cosine noise schedule:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = \cos^2\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2} \right),$$

The corresponding one-step noise variances are then recovered via $\beta_t = 1 - (\bar{\alpha}_t / \bar{\alpha}_{t-1})$. The cosine noise schedule causes the forward-process to approach noise more gradually.

Time step importance sampling

Directly optimizing L_{VLB} requires estimating a sum of per time-step losses:

$$L_{\text{VLB}} = \sum_{t=1}^{T} \mathbb{E}(L_t \mid t),$$

where $\mathbb{E}(L_t \mid t)$ denotes expectation over the data and diffusion noise at a given time point. Uniformly sampling time produces a high-variance estimator because the magnitude of L_t decreases sharply with t. To smooth the gradients, Nichol and Dhariwal [17] propose sampling time steps with probability proportional to the root second moment of the loss:

$$I \sim \text{Categorical}(p_1, \ldots, p_T), \text{ where } p_t \propto \sqrt{\mathbb{E}(L_t^2 \mid t)}.$$

Given $I = t$, define $\hat{L}_{\text{VLB}} = \frac{L_I}{p_I}$. This estimator is unbiased for L_{VLB} because:

$$\mathbb{E}(\hat{L}_{\text{VLB}}) = \sum_{t=1}^{T} \mathbb{P}(I = t)\mathbb{E}(\hat{L}_{\text{VLB}} | I = t) = \sum_{t=1}^{T} p_t \mathbb{E}\left(\frac{L_t}{p_t} \mid t\right) = L_{\text{VLB}}.$$

Choosing $p_t \propto \sqrt{\mathbb{E}(L_t^2 \mid t)}$ minimizes the variance of the loss with respect to the sampling distribution. In practice, $\mathbb{E}(L_t^2 \mid t)$ is unknown, so it is estimated online using a running history of recent observed losses at each time step.

1.6 Elucidating the Design Space

Probability flow ODE

Let $Y^*(t) \overset{d}{=} Y + \epsilon(t)$ denote the corrupted data, where $Y \sim p_{\text{data}}$ and $\epsilon(t) \sim \mathcal{N}\{0, \sigma^2(t)I\}$, with Y and $\epsilon(t)$ independent. Denote the marginal distribution of the corrupted data by $p\{y^*; \sigma(t)\}$. The probability flow ODE:

$$dY^*(t) = -\dot{\sigma}(t)\sigma(t)\nabla_{y^*} \ln p\{y^*; \sigma(t)\}\Big|_{y^*=Y^*(t)} dt$$

is a deterministic flow such that if $Y^*(t_a) \sim p\{\cdot; \sigma(t_a)\}$ then $Y^*(t_b) \sim p\{\cdot; \sigma(t_b)\}$. The probability flow ODE is a deterministic *transport equation*. Given that $Y^*(t_a) \sim p\{\cdot; \sigma(t_a)\}$, the flow specifies how $Y^*(t_a)$ evolves such that, for any target time t_b, whether before or after t_a, the evolved state satisfies $Y^*(t_b) \sim p\{\cdot; \sigma(t_b)\}$.

Derivation of the probability flow ODE

Let $p(y^*, \sigma) = \int p_{\text{data}}(y)\mathcal{N}(y^*; y, \sigma^2 I)dy$ denote the density of the corrupted data, and let $q(y^*, t) = p\{y^*, \sigma(t)\}$ denote the corrupted density at time t under time-varying noise schedule $\sigma(\cdot)$. The goal is to determine the appropriate velocity field $v(y^*, t)$,

$$dY_t^* = v(Y_t^*, t)dt, \tag{1}$$

such that if $Y_{t_a}^* \sim q(\cdot, t_a)$ at time t_a and Y_t^* evolves according to (1), then $Y_{t_b}^* \sim q(\cdot, t_b)$ at time t_b. Direct differentiation shows that $q(y^*, t)$ satisfies the heat equation:

$$\frac{\partial q(y^*, t)}{\partial t} = \dot{\sigma}(t)\sigma(t)\Delta_{y^*} q(y^*, t),$$

where Δ_{y^*} denotes the Laplacian. The quantity $q(y^*,t)v(y^*,t)$ constitutes the probability flux. $q(y^*,t)$ has units of probability mass per unit volume while $v(y^*,t)$ has units of distance per time. Thus, the probability flux has units of probability mass per unit area per time. The spatial divergence of the probability flux $\nabla_{y^*} \cdot \{q(y^*,t)v(y^*,t)\}$ has units of probability mass per unit volume per time, and by convention a positive divergence means probability mass is flowing out of the region. By conservation of (probability) mass, the local rate of probability mass accumulation $\partial_t q(y^*,t)$ must equal the spatial influx of probability mass, giving the continuity equation:

$$\frac{\partial q(y^*,t)}{\partial t} = -\nabla_{y^*} \cdot \{q(y^*,t)v(y^*,t)\}.$$

We now have two constraints $\partial_t q(y^*,t)$ must satisfy: the heat equation and the continuity equation. Equating the right hand sides:

$$\dot{\sigma}(t)\sigma(t)\Delta_{y^*} q(y^*,t) = -\nabla_{y^*} \cdot \{q(y^*,t)v(y^*,t)\}.$$

The Laplacian can be re-expressed as:

$$\dot{\sigma}(t)\sigma(t)\Delta_{y^*} q(y^*,t) = \nabla_{y^*} \cdot \{\dot{\sigma}(t)\sigma(t)\nabla_{y^*} q(y^*,t)\} = -\nabla_{y^*} \cdot \{q(y^*,t)v(y^*,t)\}.$$

Equating the quantities inside the divergence and rearranging:

$$\dot{\sigma}(t)\sigma(t)\nabla_{y^*} q(y^*,t) = -q(y^*,t)v(y^*,t),$$
$$v(y^*,t) = -\dot{\sigma}(t)\sigma(t)\frac{\nabla_{y^*} q(y^*,t)}{q(y^*,t)} = -\dot{\sigma}(t)\sigma(t)\nabla_{y^*} \ln q(y^*,t).$$

This is precisely the probability flow ODE:

$$dY_t^* = -\dot{\sigma}(t)\sigma(t)\{\nabla_{y^*} \ln q(y^*,t)\}\Big|_{y^*=Y_t^*} dt$$

Denoising as score matching

The denoising score matching objective is:

$$L_{\text{DSM}}(\theta;\sigma) = \mathbb{E}_{y\sim p_{\text{data}},\epsilon\sim\mathcal{N}(0,\sigma^2 I)}\big|\big|s_\theta(y^*;\sigma) - \nabla_{y^*} \ln p(y^*|y;\sigma)\big|\big|^2.$$

Here $s_\theta(y^*,\sigma)$ approximates the marginal score $\nabla_{y^*} \ln p(y^*;\sigma)$ of the corrupted data, and $p(y^*|y;\sigma)$ denotes the conditional distribution of the corrupted data given the input. Since $p(y^*|y;\sigma) = \mathcal{N}(y^*;y,\sigma^2 I)$, the conditional score is:

$$\nabla_{y^*} \ln p(y^*|y;\sigma) = \frac{y - y^*}{\sigma^2}.$$

It can be shown that:

$$\nabla_{y^*} \ln p(y^*;\sigma) = \frac{\mathbb{E}(y|y^*) - y^*}{\sigma^2},$$

which upon rearranging gives:

$$\mathbb{E}(y|y^*;\sigma) = y^* + \sigma^2 \nabla_{y^*} \ln p(y^*;\sigma).$$

Let $D^*(y^*;\sigma) = \mathbb{E}(y|y^*;\sigma)$ denote the optimal denoiser under squared-error loss. Reparameterize the noise-conditional score network via:

$$s_\theta(y^*;\sigma) = \frac{D_\theta(y^*;\sigma) - y^*}{\sigma^2}$$

then:

$$s_\theta(y^*;\sigma) - \nabla_{y^*} \ln p(y^*|y;\sigma) = \frac{D_\theta(y^*;\sigma) - y}{\sigma^2}$$

Therefore, the denoising score matching objective can be expressed as:

$$L_{\mathrm{DSM}}(\theta;\sigma) = \frac{1}{\sigma^4}\mathbb{E}_{y\sim p_{\mathrm{data}},\epsilon\sim\mathcal{N}(0,\sigma^2 I)}\big|\big|D_\theta(y+\epsilon;\sigma) - y\big|\big|^2,$$

which is equation (2) of Karras *et al.* [9].

EDM

Karras *et al.* [9] argue that sampling choices, including the noise schedule, ODE solver, and time discretization, are separable from the architecture and training of the denoiser. For the noise schedule, they select $\sigma(t) = t$ without dynamic scaling, which yields the simplified flow equation:

$$\frac{dY_t^*}{dt} = \frac{1}{t}\{Y_t^* - D_\theta(Y_t^*;t)\}$$

For the ODE solver, they adopt a second order explicit Runge-Kutta method, which they find provides a better balance between quality and computational expense than forward Euler (Algorithm 3). Lastly, for the time discretization, they observe that local truncation error (i.e., local error in approximating the solution to the ODE) matters more for output quality at smaller noise levels, and suggest taking smaller temporal steps as the noise level decreases.

Algorithm 3 Deterministic probability-flow sampling via the trapezoidal method

Require: Denoiser $D_\theta(y^*;\sigma)$, noise schedule $\sigma(t)$, time grid $t_0,\ldots,t_N$ with $\sigma(t_N) = 0$

Draw $Y_0^* \sim \mathcal{N}\{0,\sigma^2(t_0)I\}$
for $i = 0,\ldots,N-1$ **do**
  $\sigma_i \leftarrow \sigma(t_i), \quad \Delta_i \leftarrow t_{i+1} - t_i$
  $d_i \leftarrow \dfrac{\dot\sigma(t_i)}{\sigma_i}\{Y_i^* - D_\theta(Y_i^*;\sigma_i)\}$
  $\widetilde{Y}_{i+1}^* \leftarrow Y_i^* + \Delta_i d_i$
  if $\sigma(t_{i+1}) \neq 0$ **then**
    $\sigma_{i+1} \leftarrow \sigma(t_{i+1})$
    $d_i' \leftarrow \dfrac{\dot\sigma(t_{i+1})}{\sigma_{i+1}}\left\{\widetilde{Y}_{i+1}^* - D_\theta(\widetilde{Y}_{i+1}^*;\sigma_{i+1})\right\}$
    $Y_{i+1}^* \leftarrow Y_i^* + \Delta_i\left\{\dfrac{1}{2}d_i + \dfrac{1}{2}d_i'\right\}$
  else
    $Y_{i+1}^* \leftarrow \widetilde{Y}_{i+1}^*$
  end if
end for
return Y_N^*

Stochastic sampling

Sampling quality can sometimes be improved by injecting noise into the sampling process. Karras *et al.* [9] show that the deterministic probability flow ODE can be augmented with a Langevin diffusion term:

$$dY_t^* = -\dot{\sigma}(t)\sigma(t)s\{Y_t^*, \sigma(t)\}dt + \beta(t)\sigma^2(t)s\{Y_t^*, \sigma(t)\}dt + \sqrt{2\beta(t)}\sigma(t)dW_t,$$

where $s\{y^*, \sigma(t)\} = \nabla_{y^*} \ln p\{y^*; \sigma(t)\}$, $\beta(t) \geq 0$ controls the rate at which existing noise is replaced with new noise, and W_t is standard Brownian motion.

Denoiser parameterization

Denoiser training is facilitated by parameterizing the denoiser as:

$$D_\theta(y^*; \sigma) = c_{\text{skip}}(\sigma)y^* + c_{\text{out}}(\sigma)F_\theta\{c_{\text{in}}(\sigma)y^*; c_{\text{noise}}(\sigma)\},$$

where the $c_\bullet(\sigma)$ are known functions and F_θ is the learned network. The motivation for this parameterization is to keep the target of F_θ, specifically:

$$\frac{y - c_{\text{skip}}(\sigma)y^*}{c_{\text{out}}(\sigma)},$$

on the same scale across noise levels.

DPM-Solver

Lu *et al.* [15] introduce DPM-Solver, a family of numerical methods for efficiently sampling from pretrained diffusion models. Rather than modifying the learned model, DPM-Solver exploits the mathematical structure of diffusion probability flow ODEs to reduce numerical integration error, enabling high-quality generation with approximately 10–20 neural network evaluations.

Semilinear Structure of Diffusion ODEs

The probability flow ODE associated with a diffusion model can be expressed as

$$\frac{dX_t}{dt} = f(t)X_t + \frac{g^2(t)}{2\sigma_t}\epsilon_\theta(X_t, t),$$

where ϵ_θ is a pretrained noise-prediction network and $f(t)$, $g(t)$, and σ_t are determined by the noise schedule. This ODE is semilinear: the first term is linear in X_t, while the second is generally nonlinear.

Conventional ODE solvers approximate both components numerically. DPM-Solver instead uses the variation-of-constants formula to integrate the linear component exactly, leaving only an integral involving the nonlinear network output:

$$X_t = \frac{\alpha_t}{\alpha_s}X_s + \int_s^t \frac{\alpha_t}{\alpha_\tau}\frac{g^2(\tau)}{2\sigma_\tau}\epsilon_\theta(X_\tau, \tau)\, d\tau.$$

This eliminates discretization error associated with the known linear dynamics.

Reparameterization by Signal-to-Noise Ratio

The authors further simplify the remaining integral by introducing the half-log signal-to-noise ratio,

$$\lambda_t = \ln \frac{\alpha_t}{\sigma_t},$$

and changing the integration variable from t to λ. This yields an exact representation of the ODE solution:

$$X_t = \frac{\alpha_t}{\alpha_s} X_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda} \widehat{\epsilon}_\theta(X_\lambda, \lambda)\, d\lambda,$$

where $\widehat{\epsilon}_\theta$ denotes the noise-prediction network expressed in the new coordinate. The transformation removes the noise-schedule coefficients from the integrand, leaving a simple exponential weight multiplying the network output. Consequently, numerical approximation can focus on the network output while integrating the known exponential factor analytically.

Higher-Order Solvers and Finite Differences

The authors approximate the exponentially weighted integral by expanding the network output along the ODE trajectory in a Taylor series. Writing $E(\lambda) = \widehat{\epsilon}_\theta(X_\lambda, \lambda)$, we have

$$E(\lambda) = E(\lambda_s) + (\lambda - \lambda_s)E'(\lambda_s) + \frac{(\lambda - \lambda_s)^2}{2} E''(\lambda_s) + \cdots .$$

The polynomial terms can be integrated analytically against $e^{-\lambda}$, leaving the problem of approximating derivatives of the network output.

These derivatives can be approximated using finite differences, with coefficients obtained by matching Taylor expansions. For example, consider approximating $E'(\lambda_s)$ from evaluations at λ_s, $\lambda_s + h$, and $\lambda_s + 2h$. We seek coefficients a, b, c such that

$$E'(\lambda_s) \approx \frac{aE(\lambda_s) + bE(\lambda_s + h) + cE(\lambda_s + 2h)}{h}.$$

Expanding the network outputs around λ_s and matching the coefficients of the constant, first-derivative, and second-derivative terms gives

$$a + b + c = 0, \qquad b + 2c = 1, \qquad b + 4c = 0.$$

Solving yields $a = -3/2$, $b = 2$, and $c = -1/2$, giving the second-order forward-difference approximation

$$E'(\lambda_s) \approx \frac{-3E(\lambda_s) + 4E(\lambda_s + h) - E(\lambda_s + 2h)}{2h}.$$

The same undetermined-coefficients approach can be used to construct approximations to higher derivatives or, more directly, quadrature weights that integrate low-degree polynomials exactly under the exponential weight.

DPM-Solver follows this general principle but uses specialized exponential-integrator formulas rather than directly substituting standard finite-difference approximations. Its intermediate network evaluations are taken at approximately computed trajectory states, and their differences are combined using coefficients chosen to achieve the desired numerical accuracy. DPM-Solver-1 treats the network output as constant, DPM-Solver-2 incorporates linear variation, and DPM-Solver-3 incorporates quadratic variation, requiring one, two, and three network evaluations per step, respectively.

Rectified Flow

Liu *et al.* [14] introduce rectified flow, an ODE-based framework for transporting samples between two distributions π_0 and π_1. Given a coupling (X_0, X_1) where $X_0 \sim \pi_0$ follows a noise distribution and $X_1 \sim \pi_1$ follows the data distribution, they begin from the linear interpolation $X_t = (1-t)X_0 + tX_1$, whose pathwise velocity is $\dot{X}_t = X_1 - X_0$. This interpolation is non-causal in the sense that its velocity depends on the unknown destination X_1, so it cannot be simulated forward from the current state alone. Rectified flow instead learns a causal velocity field $v_\theta(x,t)$ that depends only on the current location and time by minimizing

$$\mathcal{L}(\theta) = \int_0^1 \mathbb{E}\left[\|(X_1 - X_0) - v_\theta(X_t, t)\|^2\right] dt.$$

The population minimizer is $v^X(x,t) = \mathbb{E}(X_1 - X_0 \mid X_t = x)$, the conditional mean velocity of all interpolation paths passing through x at time t. After training, new samples are generated by drawing $Z_0 \sim \pi_0$ and numerically integrating the learned ODE

$$Z_0 \sim \pi_0, \qquad Z_1 = Z_0 + \int_0^1 v_\theta(Z_t, t)\, dt.$$

For example, Euler integration with N steps uses:

$$Z_{(k+1)/N} = Z_{k/N} + \Delta t \cdot v_\theta(Z_{k/N}, k/N)$$

for $\Delta t = N^{-1}$ and $k = 0, \ldots, N-1$. Under exact estimation and suitable regularity conditions, the resulting ODE preserves the time-marginal distributions of the original interpolation process. Thus, the distribution of Z_t equals that of X_t, and in particular $Z_1 \sim \pi_1$.

The authors introduce *reflow*, in which the endpoint pairs (Z_0, Z_1) generated by one rectified flow are used as the training coupling for another. Specifically, after integrating the first learned ODE to obtain paired endpoints $(Z_0^{(1)}, Z_1^{(1)})$, a second velocity field is trained using the interpolations $(1-t)Z_0^{(1)} + tZ_1^{(1)}$ and targets $Z_1^{(1)} - Z_0^{(1)}$. Repeated rectification tends to reduce path conflicts and straighten the learned trajectories while preserving the endpoint distributions. In the ideal case, $Z_t = (1-t)Z_0 + tZ_1$ and the velocity is constant along each trajectory, so a single Euler step recovers Z_1 exactly. This straightening property is important computationally because it reduces ODE discretization error and enables accurate few-step or one-step generation.

To further accelerate generation, the authors introduce *distillation*, which directly approximates the endpoint mapping of a trained rectified flow. Given paired samples (Z_0, Z_1)

obtained by numerically integrating the teacher ODE, a student network T_ϕ is trained to predict Z_1 from Z_0 by minimizing

$$\mathcal{L}_{\text{distill}}(\phi) = \mathbb{E}\left[\|T_\phi(Z_0) - Z_1\|^2\right].$$

Equivalently, the student may predict the displacement $Z_1 - Z_0$, so that $T_\phi(Z_0) = Z_0 + u_\phi(Z_0)$. Once trained, the student generates a sample with a single network evaluation, without ODE integration. Unlike reflow, which learns a new transport coupling with straighter trajectories, distillation aims to reproduce the existing teacher coupling. The authors recommend distillation after reflow, when the transport trajectories are already nearly straight and admit accurate one-step approximations.

Finally, the framework extends beyond straight interpolation. For any differentiable stochastic path X_t connecting the endpoint distributions, one may train v against the pathwise velocity $\dot{X}_t$, yielding the population field $v^X(x,t) = \mathbb{E}(\dot{X}_t \mid X_t = x)$ and again preserving the marginal law of X_t under suitable regularity conditions. In particular, diffusion probability-flow ODEs and DDIM can be represented using paths of the form $X_t = \alpha_t X_1 + \beta_t X_0$, where $X_0 \sim \mathcal{N}(0, I)$, with the appropriate initialization. This perspective shows that diffusion-derived ODEs correspond to particular choices of interpolation schedule rather than fundamentally different deterministic transport mechanisms. Liu *et al.* argue that the canonical rectified-flow choice $\alpha_t = t$ and $\beta_t = 1 - t$ is preferable because it gives straight, constant-speed interpolation, whereas common diffusion-derived schedules generally induce curved trajectories and non-uniform speed.

1.7 Implementation

Training

1. Sample $X_0 \sim \pi_0$, typically from a standard Gaussian distribution.
2. Sample $X_1 \sim \pi_1$ independently from the training data.
3. Sample $t \sim \text{Uniform}(0, 1)$.
4. Construct the interpolation $X_t = (1 - t)X_0 + tX_1$.
5. Update the neural network v_θ by minimizing the squared-error loss:

$$\mathcal{L}(\theta) = \mathbb{E}\|v_\theta(X_t, t) - (X_1 - X_0)\|^2$$

Sampling

1. Draw $Z_0 \sim \pi_0$.
2. Numerically integrate $dZ_t = v_\theta(Z_t, t)dt$ from $t = 0$ to $t = 1$.
3. Return the terminal state Z_1 as the generated sample.

Reflow

1. Draw $Z_0 \sim \pi_0$ and integrate the fitted ODE to obtain Z_1, using a sufficiently accurate numerical solver.
2. Construct a dataset of paired endpoints (Z_0, Z_1).
3. Retrain the rectified-flow model using these pairs in place of the original independent samples (X_0, X_1).

Reflow tends to straighten the learned trajectories, enabling accurate sampling with fewer integration steps. Excessive reflow may accumulate estimation error.

Distillation

1. Draw $Z_0 \sim \pi_0$ and integrate the final rectified-flow ODE to obtain paired endpoints (Z_0, Z_1).
2. Train a student network T_ϕ to predict Z_1 from Z_0, minimizing

$$\mathcal{L}(\phi) = \mathbb{E}\|T_\phi(Z_0) - Z_1\|^2$$

   Equivalently, parameterize $T_\phi(Z_0) = Z_0 + u_\phi(Z_0)$ and train u_ϕ to predict the displacement $Z_1 - Z_0$.
3. Generate new samples directly as $\widehat{Z}_1 = T_\phi(Z_0)$.

Flow Matching

Lipman *et al.* [13] introduce *flow matching*, a simulation-free framework for learning generative ODEs. As in rectified flow [14], a neural network $v_\theta(x, t)$ parameterizes an ODE $dZ_t = v_\theta(Z_t, t)dt$ that transports a source distribution π_0 to a target data distribution π_1. The main generalization is to allow flexible choices of the intermediate probability path connecting these distributions.

2.1 Conditional Flow Matching

Given a prescribed probability path p_t and a velocity field u_t generating it, the flow matching objective is

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}\left[\|v_\theta(X_t, t) - u_t(X_t)\|^2\right], \qquad X_t \sim p_t, \quad t \sim \mathrm{Uniform}(0, 1).$$

However, sampling from the marginal distribution p_t and deriving its velocity field u_t may be intractable. To address this, the authors introduce *conditional flow matching*. For each data sample $X_1 \sim \pi_1$, specify a tractable conditional probability path $p_t(x \mid X_1)$ with known velocity $u_t(x \mid X_1)$. The objective becomes

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}\left[\|v_\theta(X_t, t) - u_t(X_t \mid X_1)\|^2\right], \qquad X_t \sim p_t(\cdot \mid X_1).$$

The two objectives differ only by a constant independent of θ and therefore have identical gradients. Thus, the marginal velocity field can be learned through supervised regression against tractable conditional velocities, without evaluating the marginal density or integrating an ODE during training.

2.2 Gaussian Conditional Paths

The authors consider Gaussian conditional paths of the form

$$X_t = \mu_t(X_1) + \sigma_t \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where $\mu_0(X_1) = 0$, $\sigma_0 = 1$, $\mu_1(X_1) = X_1$, and $\sigma_1 = \sigma_{\min} > 0$. The small positive terminal variance prevents the conditional distribution from degenerating into a point mass, although the resulting terminal distribution is a Gaussian-smoothed approximation to π_1. Differentiating the interpolation gives the conditional velocity target

$$u_t(X_t \mid X_1) = \dot{\mu}_t(X_1) + \dot{\sigma}_t \epsilon.$$

Training therefore consists of sampling X_1, ϵ, and t, constructing X_t, and regressing $v_\theta(X_t, t)$ against this target.

A particularly simple choice is the *conditional optimal-transport path*, with $\mu_t(X_1) = tX_1$ and $\sigma_t = 1 - (1 - \sigma_{\min})t$. This yields

$$X_t = tX_1 + \{1 - (1 - \sigma_{\min})t\}\epsilon, \qquad u_t(X_t \mid X_1) = X_1 - (1 - \sigma_{\min})\epsilon.$$

The conditional trajectories are straight and have constant velocity. As $\sigma_{\min} \to 0$, this recovers the rectified-flow objective of Liu *et al.* [14]. However, straight conditional trajectories do not necessarily imply straight trajectories of the learned marginal ODE.

2.3 Relationship to Diffusion Models

Diffusion models prescribe probability paths through a stochastic noising process, whereas flow matching provides a general objective for learning deterministic velocity fields that generate prescribed probability paths. In particular, the Gaussian conditional-path framework encompasses diffusion-derived schedules. For such paths, score matching learns the marginal score $s_t(x) = \nabla_x \log p_t(x)$, which can be used to construct the corresponding probability-flow ODE. Flow matching instead learns the ODE velocity directly, using conditional velocity regression. With matched diffusion schedules, both approaches can recover the same deterministic dynamics.

Unlike diffusion-derived formulations, flow matching also accommodates probability paths specified independently of stochastic diffusion processes. This flexibility allows the interpolation geometry to be selected for computational efficiency. Empirically, the authors find that conditional optimal-transport paths improve training and numerical sampling efficiency relative to the diffusion-based paths they evaluate.

Multisample Flow Matching

Pooladian *et al.* [18] introduce *multisample flow matching*, which extends conditional flow matching by replacing independently sampled $(\text{noise}, \text{data})$ pairs with optimal transport couplings constructed within minibatches. The motivation is that independent pairings can produce conflicting interpolation velocities, increasing gradient variance and inducing curved trajectories that require more numerical integration steps.

Minibatch Coupling

Rather than sampling $(X_0, X_1) \sim \pi_0 \times \pi_1$, the authors construct a joint distribution $q(X_0, X_1)$ with marginals π_0 and π_1. Their principal approach is minibatch optimal transport:

1. Independently sample minibatches $\{X_0^{(i)}\}_{i=1}^k \sim \pi_0$ and $\{X_1^{(i)}\}_{i=1}^k \sim \pi_1$.
2. Find the optimal one-to-one assignment between the two minibatches by minimizing the total squared Euclidean transport cost:

$$\sum_{i=1}^{k} \|X_0^{(i)} - X_1^{(i)}\|^2,$$

   where the indices of the target samples are reordered according to the assignment.
3. Using the optimally matched pairs, construct interpolations $X_t^{(i)} = (1-t)X_0^{(i)} + tX_1^{(i)}$, where $t \sim \text{Uniform}(0, 1)$.
4. Train v_θ to predict the matched displacement $X_1^{(i)} - X_0^{(i)}$ under squared-error loss.

The authors also investigate entropically regularized optimal transport and approximate matching algorithms. Because each assignment preserves the empirical source and target distributions, the resulting population coupling retains the correct marginals.

Theoretical Motivation

Improved couplings reduce ambiguity in the conditional velocity:

$$v^*(x, t) = \mathbb{E}(X_1 - X_0 \mid X_t = x)$$

potentially reducing stochastic-gradient variance and producing straighter ODE trajectories. Under suitable assumptions, the authors show that as the minibatch size increases, optimal-transport couplings yield increasingly straight marginal flows. These properties improve numerical integration efficiency without requiring iterative reflow.

An important advantage over directly regressing X_1 on X_0 is marginal preservation. Direct regression learns $\mathbb{E}(X_1 \mid X_0)$, which generally does not transport π_0 to π_1 when the coupling is stochastic. In contrast, integrating the population-optimal flow-matching velocity preserves the prescribed marginal probability path.

Empirical Findings

Experiments on ImageNet demonstrate that minibatch matching improves training efficiency, trajectory consistency, and sample quality under limited numerical integration budgets, with modest additional training cost. Notably, approximate matching methods sometimes perform comparably to or better than exact minibatch optimal transport, suggesting that minimizing transport cost is not equivalent to optimizing generative performance.

EDM2

Karras *et al.* [10] introduce EDM2, a collection of architectural and training improvements for diffusion models. The main contributions are magnitude-preserving network operations, adaptive loss weighting, and post-hoc exponential moving averaging (EMA).

Magnitude Preservation

The authors observe that weight and activation magnitudes tend to grow during training, causing changes in effective learning rates and the distributions of intermediate activations. To address this, they introduce magnitude-preserving operations throughout the network. For example, a linear layer uses normalized weights:

$$y = \frac{w^\top x}{\|w\|_2}.$$

Under approximately uncorrelated, equal-variance inputs, this preserves activation variance. Similar adjustments are made to nonlinearities, residual connections, and other operations, substantially reducing the need for activation normalization.

However, normalizing weights during the forward pass does not prevent the underlying parameters from growing. EDM2 therefore introduces **forced weight normalization**, explicitly renormalizing stored weights after optimizer updates. This stabilizes their magnitudes and prevents uncontrolled changes in effective learning rates. The authors also introduce an explicit learning-rate decay schedule to replace the implicit decay previously induced by weight growth.

Adaptive Loss Weighting

Diffusion models are trained across a range of noise levels σ, with denoising losses that can differ substantially in magnitude and evolve at different rates. Rather than relying on fixed noise-dependent weights, EDM2 learns an adaptive weighting function $u_\phi(\sigma)$:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_\sigma \left[e^{-u_\phi(\sigma)} L_\theta(\sigma) + u_\phi(\sigma) \right],$$

where $L_\theta(\sigma)$ is the conditional denoising loss. For fixed θ, minimizing over u_ϕ gives

$$u^*(\sigma) = \ln L_\theta(\sigma).$$

Thus, the effective loss weight is approximately $1/L_\theta(\sigma)$, balancing the expected contributions of different noise levels as training progresses. This extends uncertainty-based multitask loss weighting to a continuous family of denoising tasks.

Post-hoc EMA

EMA reduces variability in model parameters by averaging their values across training steps. However, its optimal timescale can depend strongly on training duration, model architecture, and sampling configuration. EDM2 introduces a power-function averaging scheme:

$$\bar{\theta}_\gamma(T) = \frac{\gamma + 1}{T^{\gamma+1}} \int_0^T \tau^\gamma \theta(\tau)\, d\tau,$$

where γ controls the emphasis on recent parameters. Unlike conventional EMA, the effective averaging window grows proportionally with training duration.

Rather than storing every model checkpoint, the authors periodically save parameter averages with different weighting profiles. By taking linear combinations of these saved averages, alternative EMA profiles can be approximately reconstructed. This enables retrospective tuning of the EMA timescale without retraining the model or storing its complete parameter history.

Scalable Interpolant Transformers

Ma *et al.* introduce Scalable Interpolant Transformers (SiTs), a framework for training transformer-based generative models by explicitly learning a continuous transport between the data and Gaussian noise distributions [16]. SiT is designed to separate several modeling choices that are entangled in standard diffusion models, including the interpolation path, the prediction target, and the sampling procedure, while retaining the same latent transformer architecture as DiT. In particular, SiT uses the same pretrained VAE, patchification, transformer blocks, and AdaLN-Zero conditioning as DiT; the primary changes are to the generative objective and sampler. Given the interpolating process

$$\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_* + \sigma_t \boldsymbol{\epsilon},$$

the model directly predicts the corresponding velocity $\dot{\boldsymbol{x}}_t = \dot{\alpha}_t \boldsymbol{x}_* + \dot{\sigma}_t \boldsymbol{\epsilon}$ by minimizing

$$\mathcal{L}_v(\theta) = \int_0^1 \mathbb{E}\left[\left\|\boldsymbol{v}_\theta(\boldsymbol{x}_t, t) - \dot{\alpha}_t \boldsymbol{x}_* - \dot{\sigma}_t \boldsymbol{\epsilon}\right\|^2\right] dt.$$

Their preferred configuration uses the linear interpolant $\alpha_t = 1 - t$ and $\sigma_t = t$, for which the regression target simplifies to $\boldsymbol{\epsilon} - \boldsymbol{x}_*$. Training otherwise follows DiT.

At inference time, the learned velocity field can be used with either a deterministic or stochastic sampler. For deterministic sampling, SiT integrates the probability-flow ODE backward from Gaussian noise using a second-order Heun solver. For stochastic sampling, the learned velocity is first converted to the corresponding score, and the reverse-time SDE is integrated with Euler–Maruyama.

5.1 Unifying diffusion perspectives

Consider the interpolating process

$$\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_* + \sigma_t \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I),$$

where $\boldsymbol{x}_* \sim p(\boldsymbol{x})$ is a data sample and the functions α_t and σ_t control the relative contributions of data and noise as t varies. $\{\boldsymbol{x}_t : t \in [0, 1]\}$ defines a path of intermediate random variables, with corresponding marginal densities $p_t(\boldsymbol{x})$. Concretely,

$$p_t(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{x}_* \sim p(\boldsymbol{x})}\{p(\boldsymbol{x} \mid \boldsymbol{x}_*)\} = \mathbb{E}_{\boldsymbol{x}_* \sim p(\boldsymbol{x})}\{\mathcal{N}(\boldsymbol{x}; \alpha_t \boldsymbol{x}_*, \sigma^2 \boldsymbol{I})\}$$

Under the convention used by SiT, $\alpha_0 = 1$, $\sigma_0 = 0$, $\alpha_1 = 0$, and $\sigma_1 = 1$, so that $\boldsymbol{x}_0 = \boldsymbol{x}_*$ follows the data distribution while $\boldsymbol{x}_1 = \boldsymbol{\epsilon}$ follows the standard Gaussian noise

distribution. Different generative-modeling frameworks correspond to different choices of the interpolation schedules α_t and σ_t.

The interpolating process induces a velocity field

$$\boldsymbol{v}(\boldsymbol{x}, t) = \mathbb{E}(\dot{\boldsymbol{x}}_t \mid \boldsymbol{x}_t = \boldsymbol{x}) = \dot{\alpha}_t \mathbb{E}(\boldsymbol{x}_* \mid \boldsymbol{x}_t = \boldsymbol{x}) + \dot{\sigma}_t \mathbb{E}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_t = \boldsymbol{x}),$$

which describes the instantaneous transport of probability along the path of marginal distributions p_t. A velocity model can be learned directly by regressing $\boldsymbol{v}_\theta(\boldsymbol{x}_t, t)$ onto the pathwise derivative $\dot{\boldsymbol{x}}_t = \dot{\alpha}_t \boldsymbol{x}_* + \dot{\sigma}_t \boldsymbol{\epsilon}$.

The velocity field is linearly related to the score of $p_t(\boldsymbol{x})$, $\boldsymbol{s}(\boldsymbol{x}, t) = \nabla_{\boldsymbol{x}} \ln p_t(\boldsymbol{x})$, through

$$\boldsymbol{s}(\boldsymbol{x}, t) = \sigma_t^{-1} \frac{\alpha_t \boldsymbol{v}(\boldsymbol{x}, t) - \dot{\alpha}_t \boldsymbol{x}}{\dot{\alpha}_t \sigma_t - \alpha_t \dot{\sigma}_t}.$$

Thus, for a fixed interpolating process, learning the velocity field is equivalent, up to a known linear transformation, to learning the score field.

This connects directly back to the original DDPM noise-prediction formulation. Since

$$\boldsymbol{s}(\boldsymbol{x}, t) = -\sigma_t^{-1} \mathbb{E}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_t = \boldsymbol{x}),$$

the population-optimal DDPM noise predictor $\boldsymbol{\epsilon}_\theta(\boldsymbol{x}, t)$ estimates $\mathbb{E}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_t = \boldsymbol{x})$ and therefore determines the score. Consequently, for a fixed interpolation path, noise prediction, score prediction, and velocity prediction are alternative parameterizations of the same underlying time-dependent transport.

5.2 Sampling

Once the velocity field has been learned, generation follows from the fact that $\boldsymbol{v}(\boldsymbol{x}, t)$ transports probability along the prescribed path of marginal densities p_t. Specifically, the density p_t satisfies the continuity equation

$$\frac{\partial p_t(\boldsymbol{x})}{\partial t} + \nabla_{\boldsymbol{x}} \cdot \{p_t(\boldsymbol{x}) \boldsymbol{v}(\boldsymbol{x}, t)\} = 0.$$

This equation describes local conservation of probability density: $p_t(\boldsymbol{x})\boldsymbol{v}(\boldsymbol{x}, t)$ is the probability flux, so probability mass moves locally with velocity $\boldsymbol{v}(\boldsymbol{x}, t)$. It follows that individual samples can be transported along this flow according to

$$\frac{d\boldsymbol{x}_t}{dt} = \boldsymbol{v}(\boldsymbol{x}_t, t),$$

which is the probability-flow ODE. If $\boldsymbol{x}_t$ evolves according to this ODE, its marginal density evolves according to the prescribed p_t. Consequently, generation can be performed by drawing $\boldsymbol{x}_1 \sim \mathcal{N}(0, I)$ and integrating the ODE backward from $t = 1$ to $t = 0$, yielding a terminal sample distributed according to $p_0 = p$.

The same sequence of marginal distributions can also be realized by the reverse-time SDE

$$d\boldsymbol{x}_t = \left[\boldsymbol{v}(\boldsymbol{x}_t, t) - \frac{1}{2} w_t \boldsymbol{s}(\boldsymbol{x}_t, t)\right] dt + \sqrt{w_t}\, d\bar{\boldsymbol{W}}_t,$$

where $w_t \geq 0$ controls the amount of stochasticity, and $\bar{\boldsymbol{W}}_t$ is reverse-time Brownian motion. The Brownian term tends to diffuse probability mass, while the score correction $-\frac{1}{2} w_t \boldsymbol{s}(\boldsymbol{x}_t, t)$ adjusts the drift so that this additional diffusion does not change the prescribed marginal density p_t. Equivalently, substituting $\boldsymbol{s}(\boldsymbol{x}, t) = \nabla_{\boldsymbol{x}} \ln p_t(\boldsymbol{x})$ into the Fokker–Planck equation for the SDE causes the score and diffusion contributions to cancel, leaving the same continuity equation as for the probability-flow ODE. Thus, the ODE and SDE follow different individual trajectories but induce the same marginal distribution at each time and both transport $\mathcal{N}(0, I)$ at $t = 1$ to the data distribution at $t = 0$. Setting $w_t = 0$ removes both the score correction and Brownian noise, reducing the SDE exactly to the probability-flow ODE.

Multimodal DiT

Esser *et al.* [3] scale rectified-flow transformers to high-resolution text-to-image generation and introduce several practical improvements to the training and architecture. A central insight is that the geometry of the interpolation path can be separated from the distribution used to sample time along that path. Standard rectified-flow training uses the linear interpolation

$$\boldsymbol{z}_t = (1 - t)\boldsymbol{x}_0 + t\boldsymbol{\epsilon},$$

with $t \sim \text{Uniform}(0, 1)$. The authors retain this straight interpolation but generalize the timestep distribution to $t \sim \pi(t)$ for a distribution π on $(0, 1)$, thereby changing how training effort is allocated along the path without changing the path itself. They find that emphasizing intermediate timesteps improves performance relative to uniform sampling, consistent with the intuition that the velocity-prediction problem is relatively easy near the boundaries $t = 0$ and $t = 1$ and more difficult when signal and noise are substantially mixed. Thus, the interpolation geometry and the sampling distribution over that geometry constitute distinct modeling choices.

For text-to-image modeling, the authors introduce MM-DiT, a multimodal extension of DiT for text-conditioned image generation. At inference time, a text prompt is encoded into conditioning representations, while the image stream is initialized from noise and is the only modality that is ultimately generated. Within each transformer block, text and image tokens use modality-specific parameter sets for several components, including normalization, modulation, projection, and MLP layers, allowing the two modalities to retain distinct internal representations. The modalities nevertheless communicate through full bidirectional self-attention: text and image tokens are concatenated for the attention operation, so queries from either modality can attend to keys and values from both. Thus, the text representation can evolve jointly with the image representation and provide rich conditioning throughout generation, even though only the image latent is sampled and decoded into the final output. At large scale and high resolution, the authors additionally apply RMS normalization to the query and key projections before

computing attention. This QK normalization controls attention-logit growth, prevents attention-entropy collapse, and stabilizes mixed-precision training.

Improving the Training of Rectified Flows

Lee *et al.* [12] propose improved training techniques for rectified flows to enable high-quality generation with one or two neural network evaluations. Their central argument is that, in high-dimensional image spaces, linear interpolations between endpoint pairs generated by the first rectified flow rarely intersect. Consequently, a single iteration of reflow may already produce nearly straight population-optimal trajectories, making further reflow iterations unnecessary.

Improved Training

The authors introduce two principal modifications to training the second rectified flow:

1. **Nonuniform interpolation sampling:** Rather than sampling the interpolation parameter uniformly, they propose a U-shaped distribution that emphasizes the endpoints, where prediction errors are empirically larger after reflow.

2. **LPIPS-Huber loss:** They replace squared-error velocity regression with a combination of pseudo-Huber loss, which reduces sensitivity to large residuals, and LPIPS, which encourages perceptually accurate image reconstruction. Because reflow produces approximately deterministic conditional velocities, alternative losses can approximately preserve the population-optimal velocity field.

Pretrained Diffusion Initialization

Rather than training the first rectified flow from scratch, the authors reuse a pretrained diffusion model to generate the endpoint pairs required for reflow. By matching signal-to-noise ratios and rescaling inputs, the pretrained model's denoising predictions can be converted into rectified-flow velocities. Integrating the resulting ODE produces paired noise and synthetic-image samples, which are used to train the second rectified flow.

Real-Data Augmentation

The authors additionally investigate incorporating real images into reflow training. Instead of starting from Gaussian noise and generating synthetic images, they integrate the pretrained teacher ODE backward from real images to obtain corresponding synthetic noise representations. Fine-tuning on these pairs improves generation quality, although the paper does not establish that this approach is generally superior to using forward-generated pairs.

Flow Map Matching

Boffi *et al.* [1] introduce Flow Map Matching (FMM), a framework for directly learning finite-time transport maps rather than instantaneous velocity fields. The approach

provides a mathematical foundation for consistency models and related methods for few-step generation, and supports both distillation from a pretrained flow model and direct training without a teacher.

Flow Maps and Average Velocity

Consider a probability flow ODE with instantaneous velocity field $v(x,t)$. The position at time t of a trajectory starting at x_s satisfies

$$x_t = x_s + \int_s^t v(x_u, u)\, du.$$

The corresponding flow map $\Phi_{s,t}(x_s) = x_t$ can therefore be expressed as

$$\Phi_{s,t}(x_s) = x_s + \int_s^t v(\Phi_{s,u}(x_s), u)\, du.$$

The flow map is completely determined by the instantaneous velocity field, assuming the ODE has a unique solution. Unlike conventional flow matching, which learns $v(x,t)$ and numerically integrates the ODE, FMM learns $\Phi_{s,t}$ directly, enabling transport over arbitrary intervals.

The flow map is closely related to the average velocity learned by MeanFlow and Shortcut Models:

$$\Phi_{s,t}(x_s) = x_s + (t-s)u(x_s, s, t).$$

Thus, all three approaches model the same finite-interval transport, differing primarily in their training objectives. The true flow map satisfies the identity $\Phi_{s,s}(x_s) = x_s$, the composition property $\Phi_{t,\tau}(\Phi_{s,t}(x_s)) = \Phi_{s,\tau}(x_s)$, and invertibility $\Phi_{t,s}(\Phi_{s,t}(x_s)) = x_s$. The composition property is precisely the self-consistency relationship exploited by Shortcut Models.

Lagrangian and Eulerian Formulations

The authors derive two differential equations characterizing the flow map. The Lagrangian formulation follows a trajectory while holding its starting position fixed:

$$\partial_t \Phi_{s,t}(x_s) = v(\Phi_{s,t}(x_s), t).$$

Given a pretrained velocity model v_θ, this motivates Lagrangian map distillation, which trains a flow-map network $\Phi^\phi_{s,t}$ by minimizing

$$\mathcal{L}_{\mathrm{LMD}}(\phi) = \mathbb{E}\left[\left\|\partial_t \Phi^\phi_{s,t}(X_s) - v_\theta(\Phi^\phi_{s,t}(X_s), t)\right\|^2\right].$$

The objective enforces agreement between the predicted destination's rate of change and the pretrained velocity field evaluated at that destination. It does not require generating teacher trajectories through numerical integration.

The alternative Eulerian formulation characterizes how the map changes with its starting time:

$$\partial_s \Phi_{s,t}(x_s) + \nabla_x \Phi_{s,t}(x_s) v(x_s, s) = 0.$$

This yields a corresponding Eulerian distillation loss, but requires spatial derivatives of the learned map. The authors find the Lagrangian objective preferable, particularly when the map has sharp spatial variations that make its Jacobian difficult to estimate or optimize.

Direct Flow Map Matching

The Lagrangian distillation objective assumes access to a pretrained marginal velocity field. To eliminate this requirement, the authors construct a training objective using conditional velocities from stochastic interpolants. For simplicity, consider the linear interpolation

$$X_t = (1-t)X_0 + tX_1, \qquad \dot{X}_t = X_1 - X_0,$$

where X_0 and X_1 are sampled from the source and target distributions. As in conventional flow matching, the marginal velocity satisfies $v(x,t) = \mathbb{E}[\dot{X}_t \mid X_t = x]$.

The difficulty is that the Lagrangian equation evaluates velocity at the predicted destination $\Phi_{s,t}(x_s)$, which generally differs from the empirical interpolation point X_t. The authors resolve this by using the learned inverse flow map to construct a starting state whose predicted destination should equal X_t:

$$\widetilde{X}_s = \Phi^{\theta}_{t,s}(X_t), \qquad \widetilde{X}_t = \Phi^{\theta}_{s,t}(\widetilde{X}_s).$$

They then minimize the direct FMM objective

$$\mathcal{L}_{\mathrm{FMM}}(\theta) = \mathbb{E}\left[\left\| \partial_t \Phi^{\theta}_{s,t}(\widetilde{X}_s) - \dot{X}_t \right\|^2 + \left\| \widetilde{X}_t - X_t \right\|^2 \right].$$

The first term matches the derivative of the learned flow map to the conditional interpolation velocity, while the second enforces inverse consistency. At the population optimum, $\widetilde{X}_t = X_t$, ensuring that the conditional velocity provides the correct regression target. This construction enables direct learning of finite-time transport without a pretrained velocity model.

In their experiments, the authors parameterize the flow map as

$$\Phi^{\theta}_{s,t}(x_s) = x_s + (t-s) u_\theta(x_s, s, t),$$

which automatically enforces the identity condition and makes the connection to MeanFlow's average-velocity parameterization explicit.

Progressive Flow Map Matching

Although the direct objective permits learning arbitrary intervals, the authors find that training is easier when restricted to shorter intervals, $|t-s| \le 1/K$. A model trained on

these intervals can generate samples by composing approximately K transport steps.

Progressive Flow Map Matching (PFMM) subsequently trains a model to approximate this multistep composition with a single evaluation. For example, with midpoint $m = (s+t)/2$, a two-step target is

$$\Phi^{\theta}_{m,t}\left(\Phi^{\theta}_{s,m}(X_s)\right).$$

A student model is trained to predict this composition directly. The procedure can use a separate teacher and student or be incorporated into single-network training with stop-gradient targets.

This is closely related to Shortcut Models, which likewise learn longer-interval transport from compositions of shorter intervals. The principal distinction is that FMM derives its direct learning objective from the Lagrangian characterization and inverse consistency, whereas Shortcut Models use finite-interval compositional consistency as the training signal. MeanFlow instead uses a differential identity relating average and instantaneous velocity. Together, these methods provide complementary formulations for learning finite-time transport.

Discrete Flow Matching

Gat *et al.* [5] extend Flow Matching to discrete data, such as token sequences. Rather than learning a continuous velocity field governing an ODE, they construct a continuous-time Markov chain (CTMC) whose transition rates transport an initial distribution over discrete sequences to the target data distribution.

Discrete Probability Paths

Let $X_t = (X_t^1, \ldots, X_t^N)$ denote a sequence of N tokens from a vocabulary of size d, and let x^i denote the token value at position i in a particular sequence x. Given source and target sequences (X_0, X_1), the authors consider conditional probability paths of the form:

$$p_t(x^i \mid X_0, X_1) = (1-\kappa_t)\delta_{X_0^i}(x^i) + \kappa_t \delta_{X_1^i}(x^i),$$

where $\kappa_0 = 0$, $\kappa_1 = 1$, and κ_t is nondecreasing. Unlike continuous interpolation, this is a mixture of point masses, so intermediate states remain discrete. A common choice initializes every position with a special mask token.

The evolving marginal distribution is generated by a CTMC. Let $u_t^i(\widetilde{x}^i, x)$ denote the probability velocity for replacing the token at position i in sequence x with $\widetilde{x}^i$. For a sufficiently small time step h, the transition rule is:

$$P(X_{t+h}^i = \widetilde{x}^i \mid X_t = x) = \mathbb{I}\{\widetilde{x}^i = x^i\} + h u_t^i(\widetilde{x}^i, x) + o(h).$$

The probability velocities must sum to zero over possible replacement tokens and be nonnegative for transitions to different tokens.

Conditional Flow Matching

As in continuous Flow Matching, the marginal probability velocity can be obtained by averaging conditional velocities:

$$u_t^i(\widetilde{x}^i, x) = \mathbb{E}\left[u_t^i(\widetilde{x}^i, x \mid X_0, X_1) \mid X_t = x\right].$$

For the masking probability path, this gives:

$$u_t^i(\widetilde{x}^i, x) = \frac{\dot{\kappa}_t}{1-\kappa_t}\left[P(X_1^i = \widetilde{x}^i \mid X_t = x) - \mathbb{I}\{\widetilde{x}^i = x^i\}\right].$$

Here $P(X_1^i = \widetilde{x}^i \mid X_t = x)$ is the posterior probability that the target token at position i is $\widetilde{x}^i$, given the current sequence. This plays a role analogous to the conditional expectation $\mathbb{E}[X_1 - X_0 \mid X_t = x]$ in continuous Flow Matching: both average over possible endpoints to determine the marginal transport dynamics.

Rather than estimating transition rates directly, the authors train a neural network to predict the target-token posterior using cross-entropy:

$$\mathcal{L}(\theta) = -\mathbb{E}\left[\sum_{i=1}^{N} \ln P_\theta(X_1^i \mid X_t)\right].$$

The learned posterior is then converted analytically into CTMC transition rates. During generation, token positions are updated in parallel according to these rates, rather than sequentially in an autoregressive order.

Schedulers and Corrector Sampling

The scheduler κ_t determines how rapidly probability moves from source to target tokens. The authors find that nonuniform schedules can improve generation quality.

For an all-mask source, the ordinary forward CTMC reveals tokens irreversibly: once a token leaves the masked state, its transition rates vanish under the exact posterior. The authors introduce **corrector sampling**, which combines forward and backward transition dynamics while preserving the prescribed marginal probability path under exact posterior estimation. This permits previously revealed tokens to be masked and regenerated using updated sequence context, enabling iterative refinement. The key distinction is that the probability path specifies the marginal distribution at each time, but does not uniquely determine individual sampling trajectories.

Shortcut Models

Frans *et al.* [4] introduce Shortcut Models, a framework for learning finite-interval transport that enables both one-step and multi-step generation using a single neural network. Unlike conventional flow matching, which predicts instantaneous velocity, Shortcut Models condition the network on the desired integration interval, allowing it to predict displacement along potentially curved trajectories.

Average Velocity and Self-Consistency

Using the notation of MeanFlow, let $u_\theta(x, t_1, t_2)$ denote the average velocity from time t_1 to t_2. The corresponding transport update is

$$X_{t_2} = X_{t_1} + (t_2 - t_1)u_\theta(X_{t_1}, t_1, t_2).$$

For the true average-velocity field, transporting directly across an interval must agree with composing two shorter transports. Taking $t_m = (t_1 + t_2)/2$, this gives the self-consistency identity

$$u(X_{t_1}, t_1, t_2) = \frac{1}{2}\left[u(X_{t_1}, t_1, t_m) + u(X_{t_m}, t_m, t_2)\right],$$

where $X_{t_m} = X_{t_1} + (t_m - t_1)u(X_{t_1}, t_1, t_m)$. The authors use this identity to construct training targets for longer intervals from predictions over shorter intervals, avoiding explicit numerical integration of a pretrained teacher ODE.

Training Objective

Shortcut Models combine two objectives: conventional flow matching for instantaneous velocities and self-consistency for average velocities over longer intervals.

For instantaneous velocity learning, the authors use standard conditional flow matching:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}\left[\|u_\theta(X_t, t, t) - (X_1 - X_0)\|^2\right],$$

where $X_t = (1 - t)X_0 + tX_1$, with X_0 sampled from the Gaussian prior and X_1 from the data distribution.

For average velocity learning, they construct a target by composing two shorter shortcuts. Letting $t_m = (t_1 + t_2)/2$, define

$$\widetilde{X}_{t_m} = X_{t_1} + (t_m - t_1)u_\theta(X_{t_1}, t_1, t_m),$$

and

$$u_{\mathrm{tgt}} = \frac{1}{2}\left[u_\theta(X_{t_1}, t_1, t_m) + u_\theta(\widetilde{X}_{t_m}, t_m, t_2)\right].$$

The self-consistency objective is

$$\mathcal{L}_{\mathrm{SC}}(\theta) = \mathbb{E}\left[\|u_\theta(X_{t_1}, t_1, t_2) - \mathrm{sg}(u_{\mathrm{tgt}})\|^2\right],$$

where sg denotes stop-gradient. As in temporal-difference learning, the target is constructed using predictions from the current model. Importantly, the intermediate state is generated by following the learned transport rather than the empirical linear interpolation.

The two objectives are optimized jointly in a single training run, without a separate teacher or progressive distillation procedure.

Training and Stabilization

The authors intermix instantaneous and average velocity learning, using 75% of training examples for conventional flow matching and 25% for self-consistency. For computational efficiency, interval lengths are restricted to a hierarchy of powers of two, ranging from 1/128 to 1. Longer-interval targets are constructed by combining predictions over the two corresponding half-intervals. All interval lengths are trained jointly rather than introduced through a curriculum.

Two additional techniques improve training stability. First, an exponential moving average (EMA) of the network parameters is used to generate self-consistency targets, reducing fluctuations in those targets. Second, weight decay mitigates the tendency of the model to reinforce unreliable predictions early in training, when self-generated targets may be inaccurate. These choices help stabilize joint training without requiring a carefully scheduled progression through interval lengths.

MeanFlow

Geng *et al.* [6] introduce MeanFlow, a framework for one-step generative modeling that learns average rather than instantaneous velocity. Unlike rectified flow, which seeks to straighten trajectories to make few-step numerical integration accurate, MeanFlow directly predicts displacement along potentially curved trajectories.

Average Velocity and the MeanFlow Identity

Conventional flow matching learns an instantaneous velocity field $v(z, t)$, which must be integrated to generate samples. MeanFlow instead models the average velocity over an interval $[r, t]$:

$$u(Z_t, r, t) = \frac{1}{t-r} \int_r^t v(Z_\tau, \tau)\, d\tau.$$

By definition, $Z_r = Z_t - (t-r)u(Z_t, r, t)$. Thus, an accurate estimate of average velocity enables one-step generation, even when the underlying trajectory is curved.

Calculating average velocity directly would ordinarily require numerical integration of the ODE, making it expensive to use as a training target. The authors instead differentiate its definition to obtain the MeanFlow identity:

$$u(Z_t, r, t) = v(Z_t, t) - (t-r)\frac{d}{dt}u(Z_t, r, t).$$

Here, the total derivative accounts for both the explicit dependence of u on t and the dependence through Z_t, which evolves according to $dZ_t/dt = v(Z_t, t)$. This differential relationship enables training without explicitly integrating the ODE.

Training Objective

The authors parameterize average velocity using a neural network u_θ. They modify the conditional flow-matching objective by introducing a model-dependent correction to the

regression target:

$$u_{\text{tgt}} = V - (t-r)\frac{d}{dt}u_\theta(Z_t, r, t), \qquad V = \epsilon - X,$$

where $Z_t = (1-t)X + t\epsilon$, with X sampled from the data distribution and ϵ from the Gaussian prior. In computing the total derivative, the conditional velocity V replaces the unknown marginal velocity.

The network is trained by minimizing

$$\mathcal{L}(\theta) = \mathbb{E}\left[\|u_\theta(Z_t, r, t) - \text{sg}(u_{\text{tgt}})\|^2\right],$$

where sg denotes stop-gradient. The target depends on the current network, so the training procedure is analogous to reinforcement learning methods that use bootstrapped targets, such as temporal-difference learning. Stop-gradient prevents differentiation through the target. When $r = t$, the correction vanishes and the objective reduces to conventional conditional flow matching. For nonzero intervals, the correction enables learning average velocities without integrating teacher trajectories.

Classifier-free Guidance

Classifier-free guidance (CFG) improves conditional generation by combining conditional and unconditional velocity fields:

$$v^{\text{cfg}}(z, t \mid c) = \omega v(z, t \mid c) + (1-\omega)v(z, t),$$

where $\omega > 1$ amplifies the influence of condition c. Ordinarily, CFG requires separate conditional and unconditional network evaluations during generation.

MeanFlow instead applies its training identity to the guided instantaneous velocity field, learning the corresponding guided average velocity directly. The resulting model incorporates guidance into a single prediction, preserving one-network-evaluation generation.

Mixed average-instantaneous targets

The authors also investigate intermixing instantaneous and average velocity learning. When $r = t$, the MeanFlow correction vanishes and the objective reduces to conventional conditional flow matching, whereas $r < t$ encourages learning average velocity over a nonzero interval. The authors suggest that this balances learning the instantaneous velocity field with propagating that information to longer intervals. Empirically, the best performance is achieved by using average velocity targets for 25% of training examples and instantaneous velocity targets for the remaining 75%.

Flow Maps via Self-Distillation

Boffi *et al.* [2] develop a unified framework for learning finite-time flow maps without a pretrained teacher. Building on Flow Map Matching [1], the authors show that three

mathematical characterizations of the flow map lead to distinct self-distillation objectives: Lagrangian, Eulerian, and Progressive. Each combines conventional flow matching for instantaneous velocities with an additional constraint for learning finite-interval transport.

Consider the probability flow ODE:

$$\frac{dx_t}{dt} = v(x_t, t),$$

with flow map $\Phi_{s,t}(x_s) = x_t$. The authors parameterize the flow map using average velocity:

$$\Phi^\theta_{s,t}(x_s) = x_s + (t-s)u_\theta(x_s, s, t).$$

This automatically enforces $\Phi^\theta_{s,s}(x_s) = x_s$. Differentiating with respect to t and evaluating at $t = s$ gives

$$\partial_t \Phi^\theta_{s,t}(x_s)\big|_{t=s} = u_\theta(x_s, s, s).$$

Thus, the network's zero-interval predictions represent instantaneous velocity, while predictions over positive-length intervals represent average velocity. The authors use conventional conditional flow matching to train the instantaneous velocity:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}\left[\|u_\theta(X_t, t, t) - (X_1 - X_0)\|^2\right],$$

where $X_t = (1-t)X_0 + tX_1$ is a linear interpolation between independent noise and data samples. The remaining challenge is to learn $u_\theta(x_s, s, t)$ for $s < t$.

Lagrangian Self-Distillation

The Lagrangian formulation follows a particle along its trajectory while holding its starting position fixed. The flow map must satisfy

$$\partial_t \Phi_{s,t}(x_s) = v(\Phi_{s,t}(x_s), t).$$

Evaluating at $t = s$ gives the **tangent condition**:

$$\partial_t \Phi_{s,t}(x_s)|_{t=s} = v(x_s, s).$$

Under the average-velocity parameterization, this becomes $u_\theta(x_s, s, s) = v(x_s, s)$. Substituting the network's instantaneous velocity estimate into the Lagrangian equation gives the Lagrangian self-distillation (LSD) objective:

$$\mathcal{L}_{\text{LSD}}(\theta) = \mathbb{E}\left[\left\|\partial_t \Phi^\theta_{s,t}(X_s) - u_\theta(\Phi^\theta_{s,t}(X_s), t, t)\right\|^2\right].$$

The objective encourages the derivative of the predicted destination to agree with the instantaneous velocity evaluated at that destination. The instantaneous velocity estimates are supervised by flow matching and serve as model-dependent targets for learning average velocities, eliminating the need for a separately pretrained teacher. Stop-gradient is applied to the instantaneous velocity target to direct learning from instantaneous to average velocity.

Eulerian Self-Distillation

The Eulerian formulation fixes the destination time t and varies the starting time s along a trajectory. Because changing s also changes the starting position x_s, the chain rule introduces a spatial derivative:

$$\partial_s \Phi_{s,t}(x_s) + \nabla_x \Phi_{s,t}(x_s) v(x_s, s) = 0.$$

Substituting the network's instantaneous velocity estimate gives the Eulerian self-distillation (ESD) objective:

$$\mathcal{L}_{\mathrm{ESD}}(\theta) = \mathbb{E}\left[\left\|\partial_s \Phi^{\theta}_{s,t}(X_s) + \nabla_x \Phi^{\theta}_{s,t}(X_s) u_\theta(X_s, s, s)\right\|^2\right].$$

Although the Lagrangian and Eulerian objectives characterize the same population flow map, ESD requires spatial derivatives of the learned map, which can produce large and unstable gradients. The authors find LSD substantially easier to optimize.

Progressive Self-Distillation

Progressive self-distillation (PSD) instead exploits the composition property of the flow map. For an intermediate time $r \in (s, t)$,

$$\Phi_{s,t}(x_s) = \Phi_{r,t}(\Phi_{s,r}(x_s)).$$

This motivates the objective

$$\mathcal{L}_{\mathrm{PSD}}(\theta) = \mathbb{E}\left[\left\|\Phi^{\theta}_{s,t}(X_s) - \Phi^{\theta}_{r,t}(\Phi^{\theta}_{s,r}(X_s))\right\|^2\right].$$

Thus, one long transport step is trained to agree with the composition of two shorter steps. The target is constructed from the current network using stop-gradient, analogously to the model-dependent targets used in temporal-difference learning.

PSD generalizes the central self-consistency principle of Shortcut Models, which use a discrete hierarchy of interval lengths and typically divide each interval at its midpoint. The authors also consider randomly sampled intermediate times. Unlike LSD, PSD relies on the accuracy of shorter-interval predictions, so errors in those predictions may propagate into longer-interval targets.

Practical Training Considerations

All three methods combine $\mathcal{L}_{\mathrm{FM}}$ with the corresponding self-distillation objective. The authors allocate 75% of training examples to instantaneous velocity learning and 25% to finite-interval self-distillation. They also extend EDM2's learned loss weighting to the two-time setting, assigning a learned weight $w(s, t)$ to the loss at each interval:

$$\mathcal{L}(\theta, w) = \mathbb{E}_{s,t}\left[e^{-w(s,t)} \ell_\theta(s, t) + w(s, t)\right].$$

Trajectory Matching

Kemper *et al.* [11] extend Discrete Flow Matching (DFM) to structured actions, which may modify multiple components simultaneously, change the size of the state, or depend on preceding actions. The central idea is to condition on complete action trajectories rather than only on source and target states, enabling simulation-free training of continuous-time Markov chains (CTMCs) over general action spaces.

Trajectory-Conditioned Flows

As in continuous Flow Matching, the goal is to learn dynamics that transport a source distribution p_0 to a target distribution p_1. In continuous Flow Matching, interpolations between sampled endpoints provide tractable conditional velocities for learning the marginal flow. Trajectory Matching uses sequences of discrete actions for the analogous purpose: these trajectories are training devices, not paths that the learned generative process must reproduce.

Given source and target states (s_0, s_n), a trajectory consists of intermediate states and actions:

$$\tau = (s_0, a_1, s_1, \ldots, a_n, s_n), \qquad s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} s_n.$$

Multiple valid trajectories may connect the same endpoints, and their distribution is specified by $\mathbb{P}(\tau \mid s_0, s_n)$. Conditional on τ, the authors assign action execution times using the order statistics of n independent Uniform$(0, 1)$ variables. If i actions have occurred by time t, the conditional transition rate is:

$$\lambda(s_i, a \mid \tau, t) = \frac{n - i}{1 - t} \mathbb{I}\{a = a_{i+1}\}.$$

Consequently, the number of completed actions satisfies $K_t \sim \text{Binomial}(n, t)$, yielding an intermediate state $S_t = s_{K_t}$. Both intermediate states and conditional rates can therefore be obtained without simulating the CTMC.

Marginal Rate Matching

Analogously to the marginal velocity field in continuous Flow Matching, the marginal action rate is the conditional expectation of the trajectory-specific rate:

$$\lambda(s, a, t) = \mathbb{E}\left[\lambda(s, a \mid \tau, t) \mid S_t = s\right].$$

A neural network $\lambda_\theta(s, a, t)$ is trained to match the conditional rates using a Bregman divergence, such as the squared Euclidean distance $D(x, y) = \|x - y\|_2^2$. This objective has the same expected gradient as matching the generally intractable marginal rates. Under sufficient model expressiveness, the learned CTMC transports p_0 to p_1. Although each conditional trajectory specifies a single next action, marginalization assigns positive rates to multiple actions.

Action Dependencies and Rao–Blackwellization

Dependencies between actions are represented by directed acyclic graphs (DAGs), with valid trajectories corresponding to topological orderings. Rather than using the rate from a single sampled trajectory, the authors propose Rao–Blackwellizing the training target by averaging over trajectories consistent with the current state and endpoints:

$$\bar{\lambda}(s, a \mid s_0, s_n, t) = \mathbb{E}\left[\lambda(s, a \mid \tau, t) \mid S_t = s, S_0 = s_0, S_1 = s_n\right].$$

This preserves the expected gradient while reducing its variance. For independent actions, the resulting rates recover conventional factorized DFM.

Exponential Residual Flows

Gourevitch *et al.* [7] introduce Exponential Residual Flows (ERFs), a deterministic sampling framework for discrete diffusion models. Unlike conventional continuous-time Markov chain (CTMC) sampling, which introduces new randomness at each transition, ERF evolves deterministically after a single random initialization. This provides a discrete analogue of the probability-flow ODE and enables consistency distillation.

Exponential Residual Flows

ERF augments the discrete sequence X_t with continuous residual variables E_t^{ij}, one for each position i and possible token j. At initialization, the residuals are sampled independently from Exp(1). Thereafter, the augmented state evolves deterministically.

Each residual acts as a clock associated with a potential token transition. To illustrate, consider a two-state system with states A and B, and let τ denote elapsed sampling time. Starting in state A, the residual $E_B(\tau)$ decreases at the reverse transition rate $\lambda_\tau^{A\to B}$:

$$E_B(\tau) = E_B(0) - \int_0^\tau \lambda_u^{A\to B}\, du,$$

where $E_B(0) \sim \text{Exp}(1)$ is the initial random threshold. The transition $A \to B$ occurs when the accumulated rate reaches this threshold:

$$\int_0^{\tau_{\text{jump}}} \lambda_u^{A\to B}\, du = E_B(0).$$

Meanwhile, $E_A(\tau)$ increases according to the forward exit rate from A. After the transition to B, the roles reverse: E_A begins decreasing, while E_B increases. The residuals are never resampled; their deterministic evolution reuses the randomness introduced at initialization.

Marginal Preservation

The authors prove that, at every fixed time t, the augmented state satisfies:

$$X_t \sim p_t, \qquad E_t^{ij} \overset{\text{iid}}{\sim} \text{Exp}(1), \qquad X_t \perp E_t.$$

Thus, ERF preserves the marginal distributions of the original CTMC while maintaining independent exponential residuals, independent of the discrete state. However, ERF does not necessarily preserve the joint distribution across time, so individual trajectories can differ substantially from those of the original stochastic process, even though their marginal distributions agree at every time point.

Consistency Distillation

Because the augmented trajectory is deterministic, ERF defines a transport map:

$$(X_s, E_s) = \Psi^{\theta}_{s,t}(X_t, E_t), \qquad s < t.$$

The authors exploit this property to extend consistency distillation to discrete diffusion models. A student network predicts a probability distribution over clean sequences from the augmented state (X_t, E_t). Given two states along the same ERF trajectory, the student is trained to make their predicted endpoint distributions agree by minimizing a KL divergence. The prediction at the earlier time s serves as a fixed target, obtained from a frozen copy of the student. This enables few-step generation without simulating the full sequence of discrete transitions.

References

[1] N M Boffi, M S Albergo, and E Vanden-Eijnden. Flow map matching with stochastic interpolants: A mathematical framework for consistency models. *arXiv*, 06 2024. https://arxiv.org/abs/2406.07507.

[2] N M Boffi, M S Albergo, and E Vanden-Eijnden. How to build a consistency model: Learning flow maps via self-distillation. *arXiv*, 05 2025. https://arxiv.org/abs/2505.18825.

[3] P Esser, S Kulal, A Blattmann, R Entezari, J Muller, H Saini, Y Levi, D Lorenz, A Sauer, F Boesel, D Podell, T Dockhorn, Z English, K Lacey, A Goodwin, Y Marek, and R Rombach. Scaling rectified flow transformers for high-resolution image synthesis. *arXiv*, 03 2024. https://arxiv.org/abs/2403.03206.

[4] K Frans, D Hafner, S Levine, and P Abbeel. One step diffusion via shortcut models. *arXiv*, 10 2024. https://arxiv.org/abs/2410.12557.

[5] I Gat, T Remez, N Shaul, F Kreuk, R T Q Chen, G Synnaeve, Y Adi, and Y Lipman. Discrete flow matching. *arXiv*, 07 2024. https://arxiv.org/abs/2407.15595.

[6] Z Geng, M Deng, X Bai, J Z Kolter, and K He. Mean flows for one-step generative modeling. *arXiv*, 05 2025. https://arxiv.org/abs/2505.13447.

[7] S Gourevitch, Y Janati, E Moulines, E P Xing, A Durmus, and G Zanella. An ode-based sampler for discrete diffusion models. *arXiv*, 10 2026. https://arxiv.org/abs/2610.40618.

[8] J Ho, A Jain, and P Abbeel. Denoising diffusion probabilistic models. *NeurIPS Proceedings*, 33:6840–6851, 06 2020. https://doi.org/10.48550/arXiv.2006.11239.

[9] T Karras, M Aittala, T Aila, and S Laine. Elucidating the design space of diffusion-based generative models. *arXiv*, 06 2022. https://arxiv.org/abs/2206.00364.

[10] T Karras, M Aittala, J Lehtinen, J Hellsten, T Aila, and S Laine. Analyzing and improving the training dynamics of diffusion models. *arXiv*, 12 2023. https://arxiv.org/abs/2312.02696.

[11] Niklas Kemper, Marten Lienen, David Lüdke, and Stephan Günnemann. Trajectory matching: Discrete flow matching with structured actions. *arXiv*, 10 2026. https://arxiv.org/abs/2610.32602.

[12] S Lee, Z Lin, and G Fanti. Improving the training of rectified flows. *arXiv*, 05 2024. https://arxiv.org/abs/2405.20320.

[13] Y Lipman, R T Q Chen, H Ben-Hamu, M Nickel, and M Le. Flow matching for generative modeling. *arXiv*, 10 2022. https://arxiv.org/abs/2210.02747.

[14] X Liu, C Gong, and Q Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv*, 09 2022. https://arxiv.org/abs/2209.03003.

[15] C Lu, Y Zhou, F Bao, et al. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *arXiv*, 06 2022. https://arxiv.org/abs/2206.00927.

[16] N Ma, M Goldstein, M S Albergo, N M Boffi, E Vanden-Eijnden, and S Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. *arXiv*, 01 2024. https://arxiv.org/abs/2401.08740.

[17] A Nichol and P Dhariwal. Improved denoising diffusion probabilistic models. *arXiv*, 02 2021. https://arxiv.org/abs/2102.09672.

[18] A-A Pooladian, H Ben-Hamu, C Domingo-Enrich, B Amos, Y Lipman, and R T Q Chen. Multisample flow matching: Straightening flows with minibatch couplings. *arXiv*, 04 2023. https://arxiv.org/abs/2304.14772.

[19] J Song, C Meng, and S Ermon. Denoising diffusion implicit models. *arXiv*, 10 2020. https://arxiv.org/abs/2010.02502.

[20] Y Song and S Ermon. Generative modeling by estimating gradients of the data distribution. *arXiv*, 07 2019. https://arxiv.org/abs/1907.05600.

[21] Y Song, J Sohl-Dickstein, DP Kingma, et al. Score-based generative modeling through stochastic differential equations. *arXiv*, 11 2020. https://arxiv.org/abs/2011.13456.